Learning and Predicting Chaotic Dynamics of the Lorenz System

NetID: DUY0341, Name: Keliang Wu

1 Introduction

The Lorenz system is a set of three coupled nonlinear ordinary differential equations originally proposed by Edward Lorenz in 1963 as a simplified model for atmospheric convection. Despite its simplicity, the system exhibits highly complex behavior and has become one of the most studied examples of chaotic dynamics in nonlinear science.

The system is defined by:

$$\frac{dx}{dt} = \sigma(y - x) \tag{1}$$

$$\frac{dy}{dt} = x(\rho - z) - y \tag{2}$$

$$\frac{dz}{dt} = xy - \beta z \tag{3}$$

where x, y, and z represent the convective intensity and the horizontal and vertical temperature gradients, respectively. The parameters σ, ρ, and β are positive constants. In this project, the standard parameter values $\sigma = 10$, $\rho = 28$, and $\beta = 8/3$ are used, under which the system is known to exhibit chaotic behavior. A defining feature of the Lorenz system is its sensitive dependence on initial conditions: two trajectories starting from nearly identical initial states diverge exponentially over time. Despite this unpredictability, the trajectories remain bounded and are attracted to a fractal structure in phase space known as the Lorenz attractor.

2 Method and Experiment Results

2.1 Numerical Simulation via RK4 and setup

2.1.1 RK4

In this project, in order to have a numerical ground truth as reference trajectories for later models, RK4 is used.

The Lorenz system can be written in vector form as:

$$\frac{d\mathbf{u}}{dt} = f(\mathbf{u}), \quad \mathbf{u} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{4}$$

where $f(\mathbf{u})$ represents the three Lorenz equations. Given the current state $\mathbf{u}_n$ at time t_n, the RK4 method advances the solution by one time step Δt as follows:

$$\mathbf{k}_1 = f(\mathbf{u}_n) \tag{5}$$

$$\mathbf{k}_2 = f\left(\mathbf{u}_n + \frac{\Delta t}{2}\mathbf{k}_1\right) \tag{6}$$

$$\mathbf{k}_3 = f\left(\mathbf{u}_n + \frac{\Delta t}{2}\mathbf{k}_2\right) \tag{7}$$

$$\mathbf{k}_4 = f(\mathbf{u}_n + \Delta t\, \mathbf{k}_3) \tag{8}$$

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \frac{\Delta t}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right) \tag{9}$$

Each $\mathbf{k}_i \in \mathbb{R}^3$ is a vector of slopes corresponding to the three state variables x, y, and z.

2.1.2 Setup

The simulation is configured as follows:

- Initial condition: $(x_0, y_0, z_0) = (1, 1, 1)$
- Time span: $t \in [0, 50]$
- Time step: $\Delta t = 0.01$
- Total steps: 5000

The resulting trajectory traces out the Lorenz attractor in three-dimensional phase space, exhibiting the characteristic chaotic behavior of the system.
Figure 1, displays the trajectory of the system based on numerical solution, which sets the ground truth of the system in this certain state. Figure 2 demonstrates the sensitivity ground truth system, which implies that the system is fundamentally difficult to predict in long-term. Figure 3 shows the numerical shifts of x, y, z in the generated data, which will also be used as reference for later methods to see the training results of ML (Machine Learning)

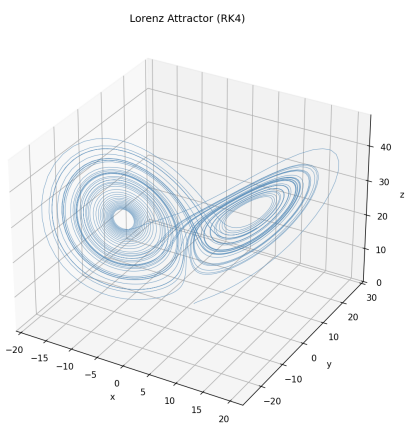

Figure 1: Numerical Solution to Lorenz System, trajectory is used as following method results references

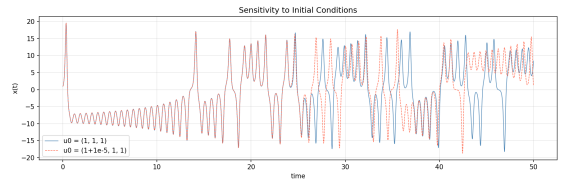

Figure 2: The numerical solution of sensitivity based on the initial condition. The trajectory will be entirely different even if the four digit on the right of the decimal point

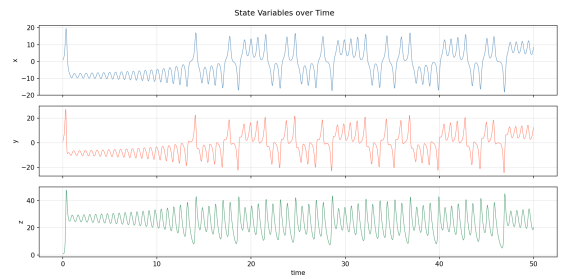

Figure 3: Shift of convective intensity (x), horizontal temperature gradient (y), vertical temperature gradient (z).

2.2 Data-Driven Neural Network

The first machine learning approach trains a standard multilayer perceptron (MLP) to learn the dynamics of the Lorenz system directly from the RK4-generated data.

2.2.1 Result: Long-term Prediction: $t \to (x, y, z)$

The first formulation trains the network to map time t directly to the system state (x, y, z). The network takes a single scalar input t and outputs three values corresponding to the state variables. Throughout the past experiences, the model structure is not always the deterministic reason of the results, I kept the MLP of both experiments with four layers, and trained with 2000 epochs. Despite the 2000 epochs, the model fails to learn any meaningful structure and collapses

to predicting a near-constant value close to the mean of the training data as shown in Figure 4. This is expected: due to the chaotic nature of the Lorenz system, the mapping $t \mapsto (x, y, z)$ is not a well-defined function, as trajectories at different times may pass through similar regions of phase space but with utterly different subsequent behavior.

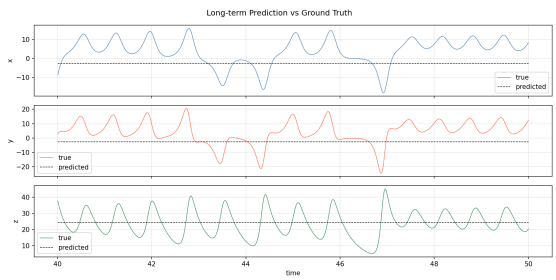

Figure 4: Long-term Plot of Prediction

2.2.2 Result: One-step Prediction: $(x, y, z)_n \to (x, y, z)_{n+1}$

The second formulation treat the problem as a one-step prediction task. Instead of mapping time to state, the network takes the current state $(x, y, z)_n$ as input and predicts the next two to three states. The network architecture consists of three hidden layers with 64 nodes each and tanh activations, trained using the Adam optimizer with MSE loss for 2000 epochs. This formulation produces significantly better results. The predicted trajectories closely follow the ground truth over the test set, demonstrating that the local dynamics of the Lorenz system can be learned effectively from data, as shown in Figure 5. However, this approach has limitations, for instance: errors accumulate over time. Each prediction feeds into the next, so small errors compound and the trajectory eventually deviates entirely from the true solution. This motivates the use of more principled approaches that explicitly model the underlying dynamics.

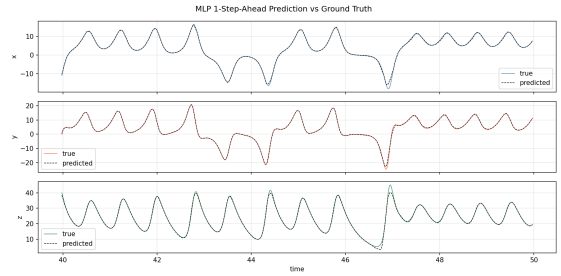

Figure 5: Short Term

2.3 Neural ODE

2.3.1 Method

Unlike the standard neural network approach, Neural ODE does not directly predict the next state. Instead, it learns the underlying dynamics of the system. It tries to learn $\frac{d\mathbf{u}}{dt} = f_\theta(\mathbf{u})$, where f_θ is a neural network. Once the network learns this, the trajectory can be recovered by numerical integration using RK4: It is basically doing 4 Euler computation and take the average of Euler results, so it is theoretically more accurate.

$$\mathbf{u}(t_1) = \mathbf{u}(t_0) + \int_{t_0}^{t_1} f_\theta(\mathbf{u})\, dt \tag{10}$$

The key difference from the previous approach is that the network is not memorizing input-output pairs — it is learning the rate of change of the system at every point in state space.

2.3.2 Setup

The network f_θ is configured as follows:

- Architecture: 3 hidden layers, 64,256,128 nodes respectively, tanh activation
- Input: state $\mathbf{u} = (x, y, z)$
- Output: derivatives $(\dot{x}, \dot{y}, \dot{z})$
- Training data: first 50 time steps, normalized
- Epochs: 3000

- Optimizer: Adam, learning rate 10^{-3}
- ODE solver: RK4 via `torchdiffeq`

2.3.3 Results

The model shows weak ability to follow the trajectory at the very beginning of the test period, but quickly diverges and collapses to a near-constant prediction. It is shown in Figure 6.
This is not surprising — the network was only trained on a short segment of the trajectory ($t \in [0, 0.49]$), and the test period is $t \in [40, 50]$, which is far outside the training range.

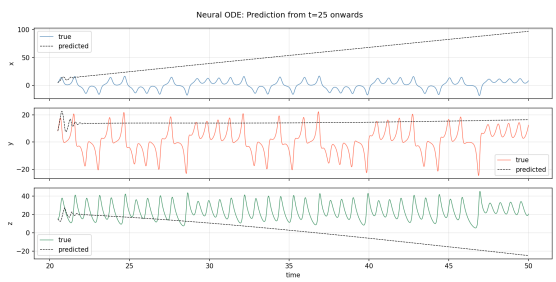

Figure 6: Prediction

Fundamentally, the Lorenz system's sensitivity to initial conditions means that even small errors in the learned dynamics compound rapidly during integration. The Neural ODE framework assumes a smooth, learnable vector field, but chaotic systems are extremely difficult to capture this way without very long training sequences and careful regularization. This result motivates the use of a physics-informed approach, where the governing equations are directly embedded into the training loss.

2.4 Physics-Informed Neural Network (PINN)

2.4.1 Method

The network takes time t as input and directly outputs the state (x, y, z). To help the network learn the oscillatory behavior of the Lorenz system, the input t is first transformed into Fourier features:

$$\phi(t) = [\sin(\omega t), \cos(\omega t), \sin(2\omega t), \cos(2\omega t), \ldots, \sin(n\omega t), \cos(n\omega t)] \quad (11)$$

The total training loss has two components:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda \mathcal{L}_{\text{phys}} \quad (12)$$

where the data loss is the standard MSE between predictions and ground truth, and the physics loss penalizes violations of the Lorenz equations:

$$\mathcal{L}_{\text{phys}} = \left(\frac{dx}{dt} - \sigma(y - x)\right)^2 + \left(\frac{dy}{dt} - x(\rho - z) + y\right)^2 + \left(\frac{dz}{dt} - xy + \beta z\right)^2 \quad (13)$$

Training is done in two stages, if follows curriculum learning : first fitting the data alone, then fine-tuning with the physics loss added.

2.4.2 Setup

- Architecture: 3 hidden layers, 64 nodes each, tanh activation
- Input: Fourier features of t with $n = 10$ frequencies
- Output: state (x, y, z)
- Training data: $t \in [0, 10]$
- Stage 1: 5000 epochs, Adam, lr $= 10^{-3}$, data loss only
- Stage 2: 5000 epochs, Adam, lr $= 10^{-4}$, data + physics loss
  Here I am only using a simple PINN, since as I tried other structures with more layers and found that structure is not the dominant reason why the performance is insufficient.

2.4.3 Results: Prediction Beyond Training Window

To evaluate whether the PINN can generalize beyond the training data, the model is trained only on $t \in [0, 10]$ and then used to predict $t \in [10, 50]$. The physics loss acts as a regularizer, encouraging the network to learn dynamics consistent with the Lorenz equations rather than simply memorizing the training trajectory.

Training proceeds in two stages:

- Stage 1: data loss only, 5000 epochs with learning rate 10^{-3}, scheduler halving every 2000 steps
- Stage 2: data loss + physics loss ($\lambda = 0.0001$), 5000 epochs with learning rate 10^{-4}

The model fits the trajectory well inside the training window. After $t = 10$, the forecast deviates steadily from the actual trajectory while maintaining the proper oscillation frequency and amplitude for a brief time. But overall is proves that PINN does learn the equation which is the fundamental computation of the Lorenz equaion, even though not perfect. Figure 7 shows the prediction results, and Figure 8 displays the 3d trajectory, even though not accurate, the point is to prove tha PINN can learn the rule of Lorenz System, despite Lorenz system can be utterly different with minimum initial condition and its chaotic nature.
The predicted trajectory partially recovers the butterfly form in the training zone, but as it extends farther in time, it loses the proper geometry, as the 3D attractor figure illustrates.

Figure 7: Prediction Results of x, y, z

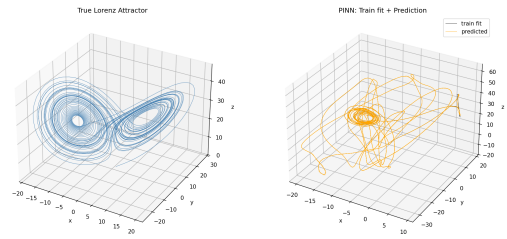

Figure 8: Prediction Results of Trajectory

3 Discussion

As the comparison in Methods and Experiments Results, a consistent observation across all four approaches is that short-term prediction is achievable, while accurate long-term trajectory prediction remains inherently difficult due to the chaotic dynamics of the Lorenz system. Because the system is extremely sensitive to initial conditions, even minor modeling errors grow exponentially over time, eventually causing all methods to deviate from the true trajectory. This limitation is most apparent in the purely data-driven MLP model. When the network is trained to learn a direct mapping from time to state, the model fails entirely, as the chaotic relationship $t \mapsto (x, y, z)$ is too irregular to approximate effectively. Reformulating the task as one-step state prediction improves performance and enables accurate local forecasts. However, since the model does not incorporate any physical structure, prediction errors accumulate rapidly during autoregressive rollout. Neural ODE attempts to address this issue by learning the underlying continuous dynamics f_θ rather than memorizing discrete transitions, which provides a more principled representation of the system. Nevertheless, when the learned dynamics are integrated over longer time horizons, prediction errors still amplify, and the model struggles to reproduce the full chaotic attractor using only a limited training window. PINNs approach the problem from a different perspective by incorporating the Lorenz equations directly into the training objective. By enforcing these governing physical con-

straints, the network is encouraged to produce solutions that remain consistent with the system dynamics even outside the observed data.

As a result, the model generates more structured trajectories and partially recovers the geometry of the Lorenz attractor beyond the training interval—an outcome not achieved by either the MLP or Neural ODE models. This advantage, however, comes at the expense of higher optimization complexity: simple joint training with both data and physics losses does not consistently converge, necessitating a two-stage training approach along with Fourier feature inputs to stabilize learning. Overall, our results show that while adding physical restrictions to chaotic system models can enhance generalization, the butterfly effect's inherent unpredictability cannot be eliminated. In the end, the system's inherent characteristics—rather than the particular modeling technique—are the main source of the constraint.